面向云存储的副本–纠删混合冗余：基于混合整数线性规划的多区域成本最优放置与迁出决策

Replica–Erasure-Coded Hybrid Redundancy for Cloud Storage: Cost-Optimal Multi-Region Placement and Eviction via MILP

作者姓名
单位院系
城市，国家，电子邮箱

摘要—云存储在多地域部署已成常态。副本冗余能够缩短读取路径并提升可用性，但存储占用与广域链路流量成本较高；纠删码可降低存储因子，却伴随分片布局、跨区修复或重构流量、以及编码与管理开销。实践中，纯副本方案与纯纠删方案往往难以在同一计价模型下同时达到成本、时延与可靠性目标的最优组合。现有研究多分别讨论副本策略或纠删参数，缺少在统一成本模型下、面向多区域访问轨迹的联合决策：哪些对象保持多副本、哪些转为纠删编码、分片如何跨区放置、以及何时迁出或留存。

本文提出一种副本–纠删混合冗余框架，将阶段性策略决策建模为混合整数线性规划（**Mixed Integer Linear Programming, MILP**），在容量、可解码性、分片（**shard**）布局一致性及迁出/保留规则等约束下，最小化存储及跨区域传输项构成的综合成本。其将轨迹统计得到的 **TTL**、对象体量及拓扑上的存储与边费单价并入 **MILP** 系数构造，由商业求解器在区域时间窗内重算布局；放置与迁出语义与请求级仿真器对齐，以便可追溯地比较总成本。在云厂商的访问轨迹上，相对纯副本及启发式基线，实验给出总成本改进，并展示在存储与流量之间的折中关系。

***Index Terms*—云存储；纠删码；多副本；混合冗余；混合整数线性规划；跨区域带宽；存储成本；放置策略；仿真评估**

I. 引言

云存储规模的持续增长与多地域容灾需求，推动服务商在副本、纠删码与跨区复制之间进行工程权衡 [1]。**副本**将完整对象复制到多个可用区或区域，有利于读就近与故障隔离，但存储放大与跨区域同步带来显性费用。**纠删码**以 k 个数据分片与 m 个校验分片（记为 (k, m)）换取低于 k 倍的对象存储开销；其代价体现为分片跨区分布、读重构或修复时的带宽聚集，以及实现与元数据管理的复杂性。

纯副本与纯纠删通常在**成本–时延–可用性**上不可同时达到最优：提高副本因子改善读与时延却增加存储；提高纠删率降低存储却可能放大跨区修复与长尾读。另一方面，工程文献常孤立设计 “副本放置启发式” 或 “纠删码宽度（k+m 分片数）”，较少在**统一成本模型**下，面向真实或合成**访问轨迹**联合优化 “谁副本化、谁纠删化、各 **shard（分片）**如何落区、何时因访问衰减而迁出”。

研究缺口概括如下：

- 缺少将**拓扑节点存储单价**与**边上跨区域 egress（placement transfer）计价**在同一线性目标中与**离散冗余形态**显式联立的可复现实例；
- 缺少在同一 MILP 中**联合择优副本放置与外传单条纹 EC（原型固定两片数据条纹）**、同时又与**仿真侧 TTL/驱逐语义**对齐的完整刻画——需注意**驱逐时机并非 MILP 的 0–1 决策向量**，而主要由

仿真依据 TTL 执行；

- 缺少从**轨迹直方图**到**系数快照**再到**写路径触发求解**的实现级描述，以便审稿者核对对象函数项与代码路径一致。

本文思路：构建**副本–纠删混合冗余框架**；在每个 **MILP epoch** 内固定**当前请求的 TTL 快照与计价拓扑**，求解**是否启用 EC**（z）以及**整副本 / 分片的放置**（x, y），最小化**线性化的存储占用项与一次性放置边费项**。**迁出与 TTL 刷新**仍在**仿真器**侧执行；轨迹更新 **TevictRanges 配对 TTL 直方图**后反向**刷新 MILP 系数**。

本文主要贡献如下：

- **Firstly**，形式化原型中与代码一致的**联合成本项与可操作线性约束**：FB（可选）下**强制基地（base）整副本进目标常数项且** x_{base}=0；**基地外至多一整副本**；EC 分支**恰好两条外传数据条纹**（parity **不单独建模为第三条 MILP 变量**）；条纹**不得共处一区**（$\sum_k y_{r,k} \le 1$）。
- **Secondly**，给出**可交由 Gurobi 求解的标准 MILP** 结构：系数快照来自**配对 TTL（向量索引含义**：pair(regions[r], `issueRegion`)）与**对象体量 /** shard $= size/K_{\text{data}}$；仿真仅在**写路径、直方图桶滚动且 histogram "warm"** 后触发求解；输出**按发起区域缓存**的布局决策。
- **Finally**，在真实或公开轨迹及多种区域规模下完成评估：相对**纯副本、纯纠删**（在实现允许时）与**启发式基线**报告**总成本**，并展示在不同权重或约束松紧下存储与流量之间的**帕累托式折中曲线**；原型实现与 `PolicyMilpECnewV3` 及汇总结果（如 `All_Results.csv`）对齐，便于复核。

本文其余部分结构如下：第 II 节综述相关工作；第 III 节给出系统模型与问题陈述；第 IV 节描述系统设计与 MILP 核心；第 V 节为实验评估；第 VI 节总结并讨论未来工作。

II. 相关工作

A. 云存储冗余与纠删码

多副本策略广泛用于提升读取性能与跨故障域可用性；纠删码从 Reed–Solomon 到局部可修复码（LRC）等在降低存储开销与修复域开销之间折中 [1]。混合系统常见做法包括对热数据保留副本、对冷数据采用纠删，以降低平均存储成本。

跨可用区或区域的放置须考虑修复与读重构带宽，以及地理分散带来的时延。现有方案多数以固定策略或分层启发式为主，**缺少与细粒度访问轨迹及云计费模型对齐的联合成本最优控制**，尤其在同时考虑副本因子与**纠删分片（shard）**跨区域布局时。

B. 数据放置、缓存与生命周期策略

多区域放置、缓存复制与基于流行度的副本扩缩已有大量研究；部分在线算法在保证竞争比的前提下调配副本 [1]。生命周期与迁出（eviction）常与 TTL、成本阈值或 LRU 变体结合。

上述工作较少在**同一优化问题**中显式纳入纠删码**分片粒度**与**跨区域 placement egress** 的联合决策；本文中 **TTL/驱逐由仿真执行，MILP 仅在快照系数层面择优** z, x, y，以实现与 `PUT/GET` 记账路径对齐。

C. 优化、MILP 与近似在存储中的应用

存储与网络中的资源分配常可归结为整数或混合整数规划。**静态放置**指在规划周期内假定对象集合与需求相对稳定，一次求解布局并在周期内沿用；实践中常采用**时间分窗**与对象聚合以降低规模。**装箱**意象指在有限容量单位（节点、区域配额）内分配不同体积对象，与容量约束天然相关；**流量工程**意象指在满足链路或预算约束下调配流量，在多区域存储中可类比为跨区带宽预算下的读修复路径选择。

MILP 适合**阶段性重规划**：在窗内解一次离散布局，窗外由系统执行。其**可扩展性**受区域数、对象聚类粒度与变量数限制；常用缓解手段包括时间窗化、对象分桶聚合、热启动（warm-start）或启发式初始解。本文诚实报告所采用的窗长、限频重算与冷启动默认策略（见第 IV 节），并讨论求解超时情形。

D. 与本文的差别

与分节文献相比，本文将**副本布局与外传单条纹 EC（两片）落区**统一写入**同一 MILP 目标与线性约束**，并以**轨迹驱动的 TTL 快照**近似**非基地副本/条纹的驻留时长；迁出事件本身不在 MILP 内枚举为布尔变量**，而是由仿真依据 TTL 与既有驱逐语义记账。**MILP 仅刻画建模范围内的存储与边上 egress**

placement cost（图上 cost）线性近似，不包含单独的 API 费线性项。

III. 问题陈述

A. 系统模型

考虑集合 $\mathcal{R}$ 个区域（Region），对象集合 $\mathcal{O}$（或由轨迹即时给出）。每一对象 $o \in \mathcal{O}$ 有逻辑大小 S_o；在纠删模式 (k, m) 下，对象被分为 k 个数据分片与 m 个校验分片（统称 n=k+m 个分片），每一分片有确定大小（如 S_o/k 或由编码率确定），并须落在指定区域以保障可解码。

每一区域 $r \in \mathcal{R}$ 具**存储单价**（仿真拓扑节点属性）；有序区域对 $\text{cost}(s, r)$ 给出**放置传输 egress 线性费率（字节加权近似下的每条边上单价属性）。与 MILP 直接相连的计价分量为 StorageCostHelper** 给出的**区域存储费**与**边** $\text{cost}(\texttt{issueRegion}, r) \times$ **传输字节体积（整副本或 shard）。读命中跨区传输、修复重构带宽及 GET API 费**仍可由 **RegionManager 等在仿真侧记账**，但不并入本节**决策 MILP 的目标向量系数构造（原型）。分片（shard）**在此 **MILP** 中指**两条数据条纹的外传单载荷块，载荷体积** $S_{\text{shard}} = S_o/K_{\text{data}}$（原型 $K_{\text{data}} = 2$）；**parity 条纹不显式出现在 MILP 决策向量中**。

B. 冗余与决策变量

MILP 内离散变量仅为冗余**形态与摆放**：全局 $z \in \{0, 1\}$ 表示**是否采纳 EC 布局**（z=1 时求解两条**数据条纹**的外传放置）；对每个区域 r 有 $x_r \in \{0, 1\}$ 表示**是否存放一整副本（可选冗余副本）**；对每个**条纹槽位** $k \in \{1, 2\}$（原型对应常量 `NUM_EC_SHARD_SLOTS`=2）及区域 r 有 $y_{r,k} \in \{0, 1\}$。**迁出不建模为 MILP 决策变量**，而是由仿真在执行 TTL 时**删除过期物理副本/条纹**。

本文**原型实现**对应 **RS** 2+1 **叙事**：parity **成本语义并入基地整副本（可选 FB）；MILP 中不对 parity 条纹单列二元变量**——与一般任意 (k, m) 纠删模型**刻意收窄以便对齐仓库代码**；扩展更大 (k, m) 可作为展望在第 VI 节讨论。

C. 工作负载与决策时点

工作负载由**请求轨迹**给出：写、读、对象键、发起区域、时间戳及对象大小分布。可基于此提取热度、地域亲和与时间局部性（指访问在时间与空间上的聚集，而非去重领域的近似块局部性）。

MILP 触发时点（原型）：仅**写路径**在 **TevictRanges 配对 TTL 直方图跨越固定时长桶**（代码常量 12 **h**）且**直方图已积累足够样本（warm gate）**后**附带求解一次**；读路径 TTL 刷新**不再触发 MILP**。**决策缓存按 `issueRegion`（当前请求的 `issue_region` 列）键控**。

D. 优化目标、约束与问题边界

目标函数（原型 MILP）为**纯线性项之和**：若启用 FB，则**强制基地整副本存储费**与 `issueRegion`→base **边费**（同城自环边上可为 0）并入**常数项** f_b；其余项为 $\sum_r s_r^{\text{rep}} x_r + \sum_{r,k} s_{r,k}^{\text{ec}} y_{r,k}$。系数 s^{rep}、s^{ec} 由 `StorageCostHelper` 根据节点 `priceStorage` 计算：**基地外副本/条纹**使用配对 TTL 快照时长；f_b **中的基地存储**使用 PUT 至记账 horizon 的时长（不经配对 TTL）。每项另加 $\texttt{edgeCost}(\texttt{issueRegion} \to r)$ 乘以对应体积（整对象 GB 或 shard GB）。**原型 MILP 不显式包含容量约束或 SLA 惩罚项**。

难点在于**整数性**、轨迹**非平稳**导致的 **TTL 快照漂移**、以及**触发 MILP 仅在少量时间点**带来的**开环分段最优**。“最优性”指**该次求解在固定快照系数下的 MILP 最优（受时间限额允许近似）**，而非**含驱逐动力学闭环的全局最优控制**。

IV. 系统设计

A. 总体流程

图 1（占位）给出**总览**：轨迹摄入 → TevictRanges 配对 TTL 直方图 →（写路径且桶滚动/warm）**构造系数快照并求解 MILP → 按 `issueRegion` 缓存**布局 → `SimulatorV2/V3` 执行 `PUT/GET`，并由仿真侧 TTL/eviction 更新物理状态与 **RegionManager** 账单。

读路径须区分：本地命中读、跨区拉取整副本、以及在 EC 模式下由**两条数据条纹载荷**参与的读取计费语义（详见仿真实现）；写路径须区分：整副本扇出、z=1 时**两枚 shard 体积** $|o|/K_{\text{data}}$ 的外传放置，以及固定基准区语义。

成本感知的副本–纠删联合 MILP（与 `GurobiEc21MilpBuilder` / `PolicyMilpECnewV3` 对齐）

【**Fig. 1 占位**】 Trace → pair-TTL histogram → （write path）MILP coefficients → per-`issueRegion` cache → simulator I/O & billing

图 1. 副本–纠删混合冗余决策与仿真闭环总览（成稿时替换为正式插图）。

变量：$z \in \{0,1\}$；$\{x_r\}_{r\in\mathcal{R}}$；$\{y_{r,k}\}_{r\in\mathcal{R},\,k\in\{1,2\}}$。

目标（最小化）

$$f_b + \sum_{r\in\mathcal{R}} s_r^{\mathrm{rep}}\, x_r + \sum_{r\in\mathcal{R}}\sum_{k=1}^{2} s_{r,k}^{\mathrm{ec}}\, y_{r,k}. \tag{1}$$

其中 $f_b, s^{\mathrm{rep}}, s^{\mathrm{ec}}$ 由 `buildCoefficients` 写入 `MilpCoefficientBundle`：若启用 **FB fixed base**，则 $s_{\mathrm{base}}^{\mathrm{rep}}$=0 **且** x_{base} **固定为** 0，基地整对象存储与 `issueRegion`→base 边费进入 f_b（时长自 **putInstant** 至 `milpDatasetEndEpochSec` 或配置 horizon）。

线性约束（摘录）——（i）若 FB：x_{base}=0；（ii）对每个区域 r：若**非（FB 且 r 为 base）**，则 $x_r + z \le 1$；若**非 FB 且 r 为 base**，则 $x_{\mathrm{base}} + z \le 2$；**FB 且 r 为 base** 时不加 x+z 绑定（强制基地副本由 f_b 吸收）；（iii）$y_{r,k} \le z$；（iv）对每个条纹槽 k：$\sum_r y_{r,k} = z$；（v）对每个区域 r：$\sum_k y_{r,k} \le z$ **且** ≤ 1（**两槽不可同区**）；（vi）$\sum_{r\neq\mathrm{base}} x_r \le 1$。非 FB 时还另有**覆盖率约束**（$\sum_r x_r + z \ge 1$ 等），详见代码分支。

消融开关：`milpEcV3AllowEc=false` 时**强制** z=0；`milpEcV3ForceEcOnly=true` 时**强制** z=1。

Algorithm 1 给出**单次求解**逻辑（不含 histogram 维护细节）。

B. 实现要点

求解器：原型采用 **Gurobi**（`GRBEnv` 常驻复用，默认时限/Gap 可由配置覆盖）。

触发与 histogram：`syncMilpIfHistogramTtlAdvanced` 仅在写路径（如 `SimulatorV2.calculateTtl`）调用；当配对 TTL 直方图按 **12 h** 桶滚动且 **histogramWarmEnoughForMilp**（与 `findMin` 同源样本阈值）为真时**调用** `runMilpForIssueRegion`。**读路径 TTL 刷新不触发 MILP**。

Algorithm 1 单次 MILP 求解流程（与原型代码对齐）

Require: 区域列表 $\mathcal{R}$；拓扑存储价与边 cost；对象模板（`LogicalObject:size`、`baseRegion`、`putInstant`）；当前请求的 `issueRegion`；TTL 向量快照 $\{\mathrm{TTL}_r\}$（由配对 $(r, \texttt{issueRegion})$ 直方图最后一桶导出）；FB 与 horizon 配置；`allowEc/forceEcOnly`

Ensure: `LayoutDecisionV3`（z、$\{x_r\}$、$\{y_{r,k}\}$、目标值），并按 `issueRegion` 写入缓存

1: **// 系数（`buildCoefficients`）**
2: 计算 $\forall r$：s_r^{rep}=$\mathrm{Stor}_r^{\mathrm{full}}$+cost(`issueRegion`, r)·$|o|_{\mathrm{GB}}$；若 FB 且 r=base 则 $s_r^{\mathrm{rep}} \leftarrow 0$
3: 计算 $\forall r, k$：$s_{r,k}^{\mathrm{ec}}$=$\mathrm{Stor}_r^{\mathrm{shard}}$ + cost(`issueRegion`, r) · $|\mathrm{shard}|_{\mathrm{GB}}$（$|\mathrm{shard}|$=$|o|/K_{\mathrm{data}}$）
4: $f_b \leftarrow$ 若 FB 则 $\mathrm{Stor}_{\mathrm{base}}^{\mathrm{full}}$(horizon) + cost(`issueRegion`, base)·$|o|_{\mathrm{GB}}$，否则 0
5: **// 建模求解（`GurobiEc21MilpBuilder.solve`）**
6: 创建二进制变量 $z, \{x_r\}, \{y_{r,k}\}$；按上文（**i**）–（**vi**）与非 FB 覆盖率分支施加线性约束
7: 若 ¬`allowEc`：附加 z=0；若 `forceEcOnly`：附加 z=1
8: 最小化 $f_b + \sum_r s_r^{\mathrm{rep}} x_r + \sum_{r,k} s_{r,k}^{\mathrm{ec}} y_{r,k}$；调用 Gurobi（时限/MIPGap 可配置）
9: **return** 舍入后的布尔布局并写入 `decisionCache`[`issueRegion`]

冷启动 / 未 warm：不进入 MILP 求解分支时布局逻辑**退化为与原 TevictRanges 策略一致的副本/TTL 路径**（论文叙述口径："不改变 histogram 冷相位的既有行为"）。

仿真对齐：`SimulatorV3.beforeWritePlacement` 刷新 MILP 缓存；`place/getPlacementWithSizes` 在 z=1 时返回**恰好两个 shard 目标区**且条纹体积为 $|o|/K_{\mathrm{data}}$；`milpStrictFail=true` 时求解失败**抛异常**以利实验复现。

可选扩展说明：`milpDatasetEndEpochSec` 可由仿真根据 trace 末端**预填**（含尾部秒数），以匹配记账 horizon。

V. 实验评估

A. 数据集

报告实际采用的访问轨迹（如类 Facebook 区域配置下的 2/3/6/9 区域实验）、预处理步骤（head 抽样、时长截断）及对比的**公平性**说明（相同轨迹、相同价格图、相同仿真语义）。

B. 评价指标

总成本为主指标，并尽可能分解为**存储、跨区域带宽、API**（若日志支持）。记录 **MILP 求解时间、成功率、重算次数**以评估可规模化程度。可选定义**有效成本比**类指标（类比模板中的 DCR 思想）：在无惩罚项时，可比作 “归一化前的费用与基线费用之比”，用于一眼比较多种窗长与区域数。

C. 数值结果与讨论

与基线对比：**Always replica**、**Always EC**（在系统可实现时）、**启发式**及本文方法 **V2/V3**（若适用）消融。参数敏感性：区域数、对象大小分布、(k, m) 或产品线、MILP 窗长与限频。

失败案例与局限：histogram 非平稳导致 TTL 快照偏差；**MILP 不含容量/API 线性项**与真实账单分项不完全一致；求解超时或数值病态；z, x, y **最优仅在快照 instantaneous 意义下成立**。

VI. 结论

本文面向多区域云存储，提出**副本–纠删混合冗余**框架：以**混合整数线性规划**在给定 **TTL 与拓扑快照**下**择优冗余形态与放置**（z, x, y），并与 **SkyStore 类轨迹仿真器**对接；**驱逐与 TTL 演化仍由仿真执行，不作为 MILP 布尔决策向量的一部分**。实验部分给出相对基线的总成本与折中行为；具体量化数值在成稿时与日志或汇总 CSV 对齐。

未来工作包括：扩展**一般** (k, m) **条纹维度**；将**读路径重构带宽或容量约束**纳入 MILP 或其外层迭代；更大规模上的**对象聚合降维**与**暖启动**；以及在更多轨迹与厂商折扣模型下验证。

致谢

参考文献

[1] 【**待补**】请替换为正式参考文献（云存储、纠删码、放置、MILP 应用等）。

[2] Ye X., Tang J., Tian W., et al., “Fast variable-grained resemblance data deduplication for cloud storage,” in *Proc. IEEE Conf.*, 2021, pp. 1–8. （版式与叙事结构参考，课题不同。）